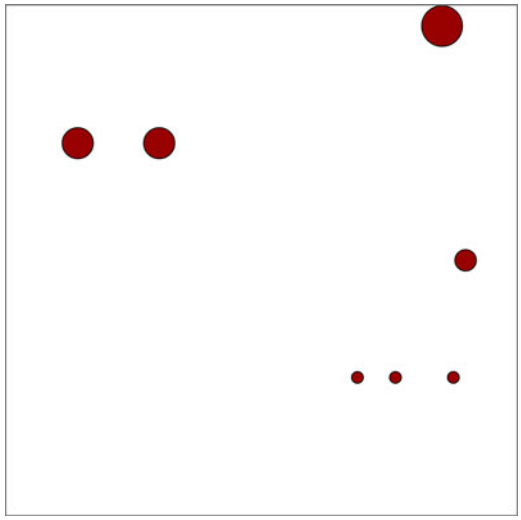


Figure 2.27 All elements corresponding to tweets that were not favorited and not retweeted were removed.

```
}).attr("cx", function(d, i) {
     return timeRamp(d.tweetTime)
}).attr("cy", function(d) {
     return 480 - yScale(d.impact)
}).style("fill", "#990000")
  .style("stroke", "black")
  .style("stroke-width", "1px");
}
```

The visual results are the same as our earlier scatterplot with the same settings, but now if we filter the array we used for the data, and bind that to the selection, we can get to the state shown in figure 2.27 by defining some useful `.exit()` behavior:

```
var filteredData = incomingData.filter(
    function(el) {return el.impact > 0}
);
d3.selectAll("circle")
  .data(filteredData, function(d) {return JSON.stringify(d)})
  .exit()
  .remove();
```

Using the stringified object won't work if you change the data in the object, because then it no longer corresponds with the original binding string. If you plan to do significant changing and updating, then you'll need a unique ID of some sort for your objects to use as your binding key.

2.4 Summary

In this chapter we looked closely at the core elements for building data visualizations using D3:

- Loading data from external files in CSV and JSON format
- Formatting and transforming data using D3 scales and built-in JavaScript functions